# Networking essentials

Reason about addresses, DNS, HTTP, TLS and firewalls well enough to debug any connectivity failure.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

## Contents

1. IP addressing and subnets (15 min)
2. DNS and name resolution (15 min)
3. HTTP and TLS (16 min)
4. Routing, gateways and NAT (16 min)
5. A layered troubleshooting toolkit (16 min)
6. Firewalls and load balancing (16 min)
7. VPNs, tunnels and private connectivity (15 min)
8. IPv6 essentials for engineers (14 min)
9. Latency, bandwidth, MTU and network performance (14 min)
10. Segmentation, security groups and least-privilege networks (13 min)
11. DHCP, ARP and the local network (12 min)
12. Debugging network problems layer by layer (15 min)
13. DNS, TLS and PKI: DNS records, resolution and caching (13 min)
14. DNS, TLS and PKI: debugging DNS failures (12 min)
15. DNS, TLS and PKI: the TLS handshake and certificate chains (13 min)
16. DNS, TLS and PKI: certificate lifecycle and automation (12 min)
17. DNS, TLS and PKI: mTLS and internal certificate authorities (13 min)
18. DNS, TLS and PKI: outage prevention checklist (11 min)
Cheat sheet

# Lesson 1. IP addressing and subnets

15 minutes. Read CIDR notation, split networks into subnets, and avoid the overlapping-range mistake that blocks cloud peering.

Your team wants to connect two cloud networks so a service in one can call a database in the other. The connection request fails with "overlapping address space". Nobody on the team can say why, because nobody has looked at the subnet ranges since the networks were created. A little subnet arithmetic prevents this class of problem entirely.

## What you will learn

- How IPv4 addresses and masks work
- How to read CIDR notation such as `10.0.1.0/24`
- How to split a network into smaller subnets
- How routes and gateways move packets between subnets

## Addresses and CIDR

An IPv4 address is 32 bits, written as four decimal numbers (`10.0.1.25`). A **network mask** says how many leading bits identify the network and how many identify the host. **CIDR notation** writes that as a slash and a number: `10.0.1.0/24` means the first 24 bits are the network.

| CIDR | Addresses | Typical use |
|---|---|---|
| `/32` | 1 | A single host |
| `/28` | 16 | A tiny subnet |
| `/24` | 256 | A common subnet size |
| `/20` | 4,096 | A larger subnet |
| `/16` | 65,536 | A whole VPC or VNet |

The number of addresses is `2^(32 - prefix)`. Smaller prefix number means a bigger network. In a classic LAN, the first address is the network address and the last is the broadcast address, so a `/24` has 254 usable hosts. Cloud providers reserve a few addresses in each subnet as well, so check your provider's rule before you plan tight ranges.

## Private ranges

Three ranges are reserved for private networks and are not routed on the public internet:

| Range | CIDR |
|---|---|
| `10.0.0.0` to `10.255.255.255` | `10.0.0.0/8` |
| `172.16.0.0` to `172.31.255.255` | `172.16.0.0/12` |
| `192.168.0.0` to `192.168.255.255` | `192.168.0.0/16` |

Note that `172.20.1.5` is private but `172.32.1.5` is public. Home routers use `192.168.x.x`, so a corporate VPN that also uses `192.168.1.0/24` will clash with your home network.

## Splitting a network

To divide `10.0.1.0/24` into four equal subnets, borrow two bits: each is a `/26` with 64 addresses. Count in steps of 64:

| Subnet | Range |
|---|---|
| `10.0.1.0/26` | `10.0.1.0` to `10.0.1.63` |
| `10.0.1.64/26` | `10.0.1.64` to `10.0.1.127` |
| `10.0.1.128/26` | `10.0.1.128` to `10.0.1.191` |

MARGA by EknathaLabs marga.eknathalabs.com

| Subnet | Range |
|---|---|
| 10.0.1.192/26 | 10.0.1.192 to 10.0.1.255 |

Two addresses are in the same subnet if they share the same network bits. `10.0.1.70` and `10.0.1.100` are both in `10.0.1.64/26`, while `10.0.1.30` is not.

The `ipcalc` tool (or Python's `ipaddress` module) checks your arithmetic:

```
$ python3 -c "import ipaddress; n=ipaddress.ip_network('10.0.1.0/24'); print([str(s) for s in n.su \
    bnets(new_prefix=26)])"
['10.0.1.0/26', '10.0.1.64/26', '10.0.1.128/26', '10.0.1.192/26']
```

## Routes and gateways

A host sends a packet directly when the destination is in its own subnet. Otherwise it sends the packet to its **default gateway**, a router that knows the way onward.

```
$ ip addr show eth0 | grep 'inet '
    inet 10.20.0.15/24 brd 10.20.0.255 scope global eth0
$ ip route
default via 10.20.0.1 dev eth0
10.20.0.0/24 dev eth0 proto kernel scope link src 10.20.0.15
$ ip route get 10.20.4.9
10.20.4.9 via 10.20.0.1 dev eth0 src 10.20.0.15
```

This is sample output. `ip route get` tells you exactly which path the kernel will use for a destination, which settles most "why does it go that way" questions. Here `10.20.4.9` is outside `10.20.0.0/24`, so the packet goes to the gateway `10.20.0.1`.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How IP addressing and subnets fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Addresses and CIDR** | An IPv4 address is 32 bits, written as four decimal numbers (10.0.1.25). | Forgetting that cloud providers reserve addresses in each subnet. |
| **Private ranges** | Three ranges are reserved for private networks and are not routed on the public internet. | Using the default range for every new network. |
| **Splitting a network** | To divide 10.0.1.0/24 into four equal subnets, borrow two bits: each is a /26 with 64 addresses. | Confusing the subnet mask with the gateway address. |
| **Routes and gateways** | A host sends a packet directly when the destination is in its own subnet. | Check the Common mistakes list below this diagram for what usually goes wrong here. |

Flow: Addresses and CIDR to Private ranges / Private ranges to Splitting a network / Splitting a network to Routes and gateways

## Incident: peering fails because the ranges overlap

> **Incident:** *Team A built a network on* `10.0.0.0/16`*. Team B built theirs on* `10.0.0.0/16` *too, because it was the default in the console. Peering is rejected.*

Routing works by matching a destination to exactly one network. If both sides contain `10.0.4.20`, a router cannot know which is meant. Compare the ranges before connecting:

```
$ python3 -c "import ipaddress as i; print(i.ip_network('10.0.0.0/16').overlaps(i.ip_network('10.0 \
    .0.0/16')))"
True
```

There is no clean fix for an existing overlap short of re-addressing one network, which usually means rebuilding everything inside it, or adding address translation, which is complex and brittle. The prevention is cheap: **keep one shared plan of address ranges** for every network, VPN and office, and allocate from it. For example, give each environment its own `/16`: `10.10.0.0/16` for production, `10.20.0.0/16` for

MARGA by EknathaLabs marga.eknathalabs.com

staging, `10.30.0.0/16` for shared services.

## Predict the subnet

## Common mistakes

- Using the default range for every new network
- Making subnets too small to grow, or a `/16` for a handful of hosts
- Forgetting that cloud providers reserve addresses in each subnet
- Confusing the subnet mask with the gateway address

### Check yourself

**How many addresses are in a /26 network?**

Answer: 64

**Which of these addresses is private?**

Answer: 172.20.1.5

**A host at 10.20.0.15/24 sends a packet to 10.20.4.9. Where does it go first?**

Answer: To its default gateway, because the destination is outside its own subnet

**Why does peering two networks with the same 10.0.0.0/16 range fail?**

Answer: The same address could exist on both sides, so routing cannot choose where to send traffic

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 2. DNS and name resolution

15 minutes. Understand how names become addresses, read dig output, and plan DNS changes so users do not hit the old server.

You migrate the shop to new servers and update its DNS record. You test, it works, and you decommission the old host. An hour later, support reports that some customers get errors while others are fine. The new address is correct. The problem is that part of the world still remembers the old one.

## What you will learn

- How a name lookup travels from your laptop to the answer
- The common record types
- How TTL and caching decide when a change becomes visible
- How to query DNS with `dig`

## How a lookup works

Your application asks the operating system, which asks a **stub resolver** configured in `/etc/resolv.conf` or by `systemd-resolved`. That forwards the question to a **recursive resolver**, often run by your ISP, your company or a public service. The recursive resolver does the legwork and caches the answer:

1. Ask a **root server** where to find `.com`
2. Ask a `.com` **server** where to find `example.com`
3. Ask the **authoritative server** for `example.com`, which owns the real answer

Each answer carries a **TTL** (time to live) in seconds. Resolvers keep the answer for that long before asking again.

## Record types

| Type | Maps | Example |
|---|---|---|
| `A` | Name to IPv4 address | `shop.example.com. A 203.0.113.10` |
| `AAAA` | Name to IPv6 address | `shop.example.com. AAAA 2001:db8::10` |
| `CNAME` | Name to another name (an alias) | `www CNAME shop.example.com.` |
| `MX` | Mail servers for a domain | `example.com. MX 10 mail.example.com.` |
| `TXT` | Free text: SPF, domain verification | `"v=spf1 include:_spf.example.com ~all"` |
| `NS` | Which servers are authoritative | `example.com. NS ns1.example.net.` |

A CNAME cannot coexist with other records on the same name, which is why the bare domain (the zone apex) usually uses an `A` record or a provider-specific alias type.

## Querying with dig

```
$ dig shop.example.com A +noall +answer
shop.example.com.   3600    IN  A   203.0.113.10
$ dig +short shop.example.com
203.0.113.10
$ dig @8.8.8.8 shop.example.com +noall +answer
shop.example.com.   3600    IN  A   203.0.113.10
$ dig +trace shop.example.com | tail -3
shop.example.com.   3600    IN  A   203.0.113.10
;; Received 61 bytes from 198.51.100.53#53(ns1.example.net) in 22 ms
```

MARGA by EknathaLabs marga.eknathalabs.com

This is sample output using documentation-reserved addresses. The second number in each answer (3600) is the **remaining TTL**. Asking your local resolver twice shows it counting down, which proves the answer came from cache. `@8.8.8.8` sends the question to a specific resolver, which lets you compare what different resolvers believe. `+trace` walks the hierarchy yourself, bypassing caches, so it shows what the authoritative servers say right now.

Locally, check what the machine uses:

```
$ resolvectl query shop.example.com
shop.example.com: 203.0.113.10                -- link: eth0
$ getent hosts shop.example.com
203.0.113.10    shop.example.com
```

`getent hosts` follows the same path as most applications, including `/etc/hosts`, so use it when `dig` and the application disagree.

## Incident: half the users still reach the old server

> **Incident:** *The `A` record for `shop.example.com` changed from `198.51.100.20` to `203.0.113.10`. Some users still land on the old host, which was switched off.*

Query the authoritative servers directly, then a public resolver, and look at the TTL:

```
$ dig +trace +noall +answer shop.example.com
shop.example.com.   86400   IN  A   203.0.113.10
$ dig @1.1.1.1 shop.example.com +noall +answer
shop.example.com.   61310   IN  A   198.51.100.20
```

The authoritative answer is correct and has a TTL of **86400 seconds (24 hours)**. The public resolver still serves the old address with about 17 hours left. Anyone behind that cache will keep hitting the old server until it expires. You cannot force other people's resolvers to forget.

The fix for now is to bring the old server back, or at least a redirect, until the TTL passes. The fix for next time is a planned sequence:

1. Days before the change, **lower the TTL** to 60 or 300 seconds and wait for the old TTL to expire.
2. Make the change.
3. Leave the old server running until traffic stops.
4. Raise the TTL again afterwards.

> **Tip:** *Browsers, the operating system and some applications cache DNS too. A long-running Java process may cache a lookup for the life of the process. If one machine keeps using an old address, restart the application or check its DNS cache settings.*

### Diagram: How a name becomes an address

| Part | What it does | What goes wrong |
|---|---|---|
| **Application** | Asks the system resolver for www.example.com. | A wrong resolv.conf means the question goes to the wrong server. |
| **Recursive resolver** | Checks its cache, then walks the tree on your behalf. | A stale cache keeps giving an old address until the TTL ends. |
| **Root and TLD servers** | Point to the servers for .com and then for example.com. | If they cannot be reached, only cached names still work. |
| **Authoritative server** | Holds the real records and answers with an A or CNAME and a TTL. | A wrong record here is the source of truth for everyone. |
| **Answer and cache** | The resolver caches the answer and returns it. | A very long TTL makes a change slow to spread. |

Flow: Application to Recursive resolver / Recursive resolver to Root and TLD servers / Recursive resolver to Authoritative server / Root and TLD servers to Answer and cache / Authoritative server to Answer and cache

## Common mistakes

MARGA by EknathaLabs marga.eknathalabs.com

- Switching off the old server immediately after a DNS change
- Testing only from your own network, which uses one cached view
- Putting a CNAME on the zone apex
- Forgetting that `/etc/hosts` overrides DNS for that machine

## Check yourself

**What does the TTL on a DNS record control?**

Answer: How long resolvers may cache the answer before asking again

**Which command bypasses caches and walks from the root to the authoritative servers?**

Answer: dig +trace name

**You are changing a record that has a 24-hour TTL. What should you do days beforehand?**

Answer: Lower the TTL and wait for the old value to expire

**Which record type points one name at another name?**

Answer: CNAME

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 3. HTTP and TLS

16 minutes. Read HTTP requests and status codes, time each phase of a request with curl, and diagnose certificate failures.

At 06:00 the monitoring dashboard turns red and customers report "your connection is not private". Nothing was deployed overnight. A certificate expired, and every client that checks it now refuses to talk to your service. Knowing how HTTP and TLS fit together lets you spot that in seconds and prevent it from recurring.

## What you will learn

- The shape of an HTTP request and response
- Status code families and common headers
- How TLS authenticates a server
- How to inspect certificates and time requests with `curl` and `openssl`

## HTTP in one page

HTTP is a text protocol: the client sends a request, the server sends a response.

```
$ curl -sv https://shop.example.com/health -o /dev/null 2>&1 | grep -E '^[<>]'
> GET /health HTTP/2
> Host: shop.example.com
> User-Agent: curl/8.5.0
> Accept: */*
< HTTP/2 200
< content-type: application/json
< cache-control: no-store
```

Lines starting with `>` are sent, and lines with `<` are received. This is sample output. The request has a **method** (`GET`), a **path**, and **headers** such as `Host`, which lets one IP address serve many sites.

| Family | Meaning | Examples |
|---|---|---|
| 2xx | Success | `200 OK`, `201 Created`, `204 No Content` |
| 3xx | Redirect | `301` permanent, `302` temporary, `304 Not Modified` |
| 4xx | The client did something wrong | `400`, `401` not authenticated, `403` forbidden, `404`, `429` rate limited |
| 5xx | The server failed | `500`, `502` bad gateway, `503` unavailable, `504` gateway timeout |

For operations, the useful split is 4xx versus 5xx. A rise in 5xx means your service is failing, and a rise in 4xx often means a bad client or a broken release of the client. `502` and `504` come from a proxy or load balancer and mean the upstream behind it failed or was too slow.

## Time every phase with curl

```
$ curl -s -o /dev/null -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect} f \
    irstbyte=%{time_starttransfer} total=%{time_total}\n' https://shop.example.com/
dns=0.012 connect=0.031 tls=0.094 firstbyte=0.842 total=0.861
```

The values are cumulative seconds. Here DNS took 12 ms, the TCP connection was ready at 31 ms, and TLS finished at 94 ms. The big jump is from 94 ms to 842 ms: the server spent about 750 ms thinking before sending the first byte. That points at the application, not the network. This one command separates "slow network" from "slow backend".

## What TLS does

TLS gives you three things: **encryption** so others cannot read traffic, **integrity** so it cannot be altered, and **authentication** so you know you reached the real server.

MARGA by EknathaLabs
marga.eknathalabs.com

Authentication uses a **certificate**, a signed statement that a public key belongs to a name. During the handshake the server presents its certificate and any intermediate certificates. The client checks that:

1. The name it asked for matches the certificate
2. The certificate is within its validity dates
3. The chain leads to a **certificate authority (CA)** the client trusts

`SNI` (Server Name Indication) is the name the client sends so a server hosting many sites can choose the right certificate. That is why you must pass the name when testing by IP.

```
$ echo | openssl s_client -connect shop.example.com:443 -servername shop.example.com 2>/dev/null | \
     openssl x509 -noout -subject -issuer -dates
subject=CN = shop.example.com
issuer=C = US, O = Let's Encrypt, CN = R11
notBefore=Aug 26 03:14:07 2026 GMT
notAfter=Nov 24 03:14:06 2026 GMT
```

Here the certificate is valid until November 24. Many ACME-issued certificates last about 90 days or less and are renewed automatically, so automation failure is the most common cause of an expiry.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How HTTP and TLS fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **HTTP in one page** | HTTP is a text protocol: the client sends a request, the server sends a response. | Testing HTTPS by IP address without sending the right SNI name. |
| **Time every phase with curl** | The values are cumulative seconds. Here DNS took 12 ms, the TCP connection was ready at 31 ms, and TLS finished at 94 ms. | Forgetting to reload the server after a certificate is renewed. |
| **What TLS does** | TLS gives you three things: encryption so others cannot read traffic, integrity so it cannot be altered, and authentication so you know you reached the real server. | Serving the leaf certificate without the intermediate chain. |

Flow: HTTP in one page to Time every phase with curl / Time every phase with curl to What TLS does

## Incident: every client fails at 06:00

**Incident:** *Health checks and customers all fail with a certificate error at the same moment.*

```
$ curl -sS https://shop.example.com/health
curl: (60) SSL certificate problem: certificate has expired
$ echo | openssl s_client -connect shop.example.com:443 -servername shop.example.com 2>/dev/null | \
     openssl x509 -noout -enddate
notAfter=Oct 24 05:59:59 2026 GMT
```

The certificate expired a minute ago. Work through it in order. First, renew it: run your certificate manager (for example `certbot renew`, or the equivalent for your CA) and reload the web server so it picks up the new files. Then verify from the outside, not from the server's own config files:

```
$ sudo certbot renew --quiet && sudo systemctl reload nginx
$ echo | openssl s_client -connect shop.example.com:443 -servername shop.example.com 2>/dev/null | \
     openssl x509 -noout -enddate
notAfter=Jan 22 06:01:12 2027 GMT
```

Next, find out *why* automatic renewal did not work. Typical causes are a blocked validation port, an expired API credential for DNS validation, or a renewal timer that was never enabled. Finally, add **monitoring for days until expiry** (alert at 21 and 7 days) so the next failure pages you well before users notice. Renewed files do nothing until the server reloads them, so check the reload step too.

## Common mistakes

MARGA by EknathaLabs
marga.eknathalabs.com

- Testing HTTPS by IP address without sending the right SNI name
- Forgetting to reload the server after a certificate is renewed
- Serving the leaf certificate without the intermediate chain
- Relying on email warnings from the CA instead of your own expiry monitoring

## Check yourself

**A load balancer returns 502 Bad Gateway. What does it indicate?**

Answer: The proxy could not get a valid response from the upstream server

**In the curl timing output, time_connect is 0.031 and time_starttransfer is 0.842. What is the most likely bottleneck?**

Answer: The server's processing time before it sends the first byte

**What does SNI allow?**

Answer: A server hosting many sites to present the certificate for the requested name

**A certificate was renewed on disk but clients still see the expired one. What step was probably missed?**

Answer: Reloading the web server so it loads the new certificate

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 4. Routing, gateways and NAT

16 minutes. Read routing tables, follow a packet across subnets, understand default gateways, source and destination NAT, and debug asymmetric or missing routes.

Addressing says who a host is. **Routing** decides where its packets go next. Almost every "cannot reach X" problem across subnets, VPNs or clouds comes down to a missing route, a wrong gateway or a NAT surprise.

## What you will learn

- How a host chooses a route using longest-prefix match
- Reading and changing routes with `ip route`
- Static versus dynamic routing at a high level
- Source NAT, destination NAT and connection tracking

## The host routing table

When a host sends a packet it asks one question: do I have a route for this destination? The most specific matching route wins, which is called **longest prefix match**.

```
$ ip route show
default via 10.0.1.1 dev eth0 proto dhcp metric 100
10.0.1.0/24 dev eth0 proto kernel scope link src 10.0.1.25
172.16.0.0/16 via 10.0.1.254 dev eth0
$ ip route get 172.16.4.9
172.16.4.9 via 10.0.1.254 dev eth0 src 10.0.1.25
$ ip route get 8.8.8.8
8.8.8.8 via 10.0.1.1 dev eth0 src 10.0.1.25
```

Destinations on the local subnet are sent directly. Everything else goes to a **gateway**, a router that forwards it on. The `default` route (0.0.0.0/0) catches traffic with no more specific match. `ip route get` is the best debugging tool: it shows the exact decision the kernel will make for one address.

## Changing routes

```
$ sudo ip route add 192.168.50.0/24 via 10.0.1.254
$ sudo ip route del 192.168.50.0/24
$ sudo ip route replace default via 10.0.1.1 dev eth0
$ ip -br addr
$ ip neigh show
```

Changes made with `ip route` vanish at reboot. Persist them through your distribution's network configuration (netplan, NetworkManager, systemd-networkd) or, in the cloud, through route tables attached to subnets.

## Routers and forwarding

A Linux host becomes a router when forwarding is enabled and it has interfaces in more than one network:

```
$ sysctl net.ipv4.ip_forward
$ sudo sysctl -w net.ipv4.ip_forward=1
$ traceroute -n 172.16.4.9
$ mtr -rn -c 10 172.16.4.9
```

Within one organisation, routers learn routes from each other with protocols such as OSPF, and between organisations with BGP. In the cloud you rarely run these yourself; you write **route tables**: entries like "10.2.0.0/16 goes to the peering connection" or "0.0.0.0/0 goes to the NAT gateway".

## NAT

**Network Address Translation** rewrites addresses as packets pass through a device. Private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) are not routable on the internet, so private hosts reach the internet through **source NAT** (also called masquerade), which replaces their address with the gateway's public one and remembers the mapping in a connection-tracking table.

MARGA by EknathaLabs
marga.eknathalabs.com

```
$ sudo iptables -t nat -A POSTROUTING -s 10.0.0.0/16 -o eth0 -j MASQUERADE
$ sudo nft list ruleset | head -20
$ sudo conntrack -L | head -5
$ sudo conntrack -C
```

**Destination NAT** (port forwarding) does the reverse: traffic arriving at a public address and port is rewritten to a private host. Cloud load balancers and Kubernetes `NodePort` services do this for you.

NAT has consequences. Inbound connections cannot start unless a rule maps them. The tracking table has a finite size, so a host making huge numbers of connections can exhaust it. Many clients behind one NAT address look like a single client to rate limiters.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How Routing, gateways and NAT fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **The host routing table** | When a host sends a packet it asks one question: do I have a route for this destination? | Debugging with ping alone when ICMP is filtered. |
| **Changing routes** | Changes made with ip route vanish at reboot. | Forgetting the return path: a route exists one way but not back. |
| **Routers and forwarding** | A Linux host becomes a router when forwarding is enabled and it has interfaces in more than one network. | Overlapping CIDR ranges between networks you later want to connect. |
| **NAT** | Network Address Translation rewrites addresses as packets pass through a device. | Not allowing for NAT when an application reports client addresses. |

Flow: The host routing table to Changing routes / Changing routes to Routers and forwarding / Routers and forwarding to NAT

## Incident: the new subnet that cannot reach the internet

> **Incident:** *A team creates a private subnet for a new service. Instances there can reach the database but every call to an external API times out. The same code works in the older subnet.*

Check what the route table says for an internet destination:

```
$ ip route get 1.1.1.1
1.1.1.1 via 10.0.9.1 dev eth0 src 10.0.9.14
$ curl -sS -m 5 -o /dev/null -w "%{http_code}\n" https://api.example.com
curl: (28) Connection timed out after 5001 milliseconds
```

The host has a default route to its subnet gateway, so the problem is upstream: the new subnet's cloud route table has no `0.0.0.0/0` entry pointing at the NAT gateway. The old subnets had one. Add the route and re-test. Infrastructure as code prevents this class of error: define a subnet and its route table together in one module.

## Common mistakes

- Forgetting the return path: a route exists one way but not back
- Overlapping CIDR ranges between networks you later want to connect
- Assuming `ip route add` persists across reboots
- Not allowing for NAT when an application reports client addresses
- Debugging with ping alone when ICMP is filtered

### Check yourself

**How does a host choose between several matching routes?**

Answer: The one with the longest (most specific) prefix

MARGA by EknathaLabs marga.eknathalabs.com

**Which command shows the routing decision for one destination?**

Answer: ip route get ADDRESS

**What does source NAT (masquerade) do?**

Answer: Rewrites private source addresses to a public one for outbound traffic

**Why do overlapping CIDR ranges cause trouble?**

Answer: The router cannot tell which network a destination belongs to when you connect them

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 5. A layered troubleshooting toolkit

16 minutes. Debug "service A cannot reach service B" in order, and learn why "connection refused" and "timed out" point to different causes.

The orders service cannot reach its database. The error says only "connection failed". You can guess for an hour, or you can test one layer at a time and know at each step what is working. This lesson gives you the order and the tools.

## What you will learn

- A fixed order for testing connectivity
- The difference between refused, timed out and unreachable
- How to test a TCP port without a client library
- How to confirm packet loss with `tcpdump`

## Work from the bottom up

Test each layer and stop at the first failure. Everything above a failure is irrelevant until it is fixed.

| Step | Question | Tool |
|---|---|---|
| 1 | Does my host have an address and a route? | `ip addr`, `ip route get` |
| 2 | Can I resolve the name? | `dig +short`, `getent hosts` |
| 3 | Can I reach the host at all? | `ping`, `mtr` |
| 4 | Is the port open on that host? | `nc -zv`, `ss -tlnp` on the server |
| 5 | Does the application answer? | `curl -v`, the real client |

Note that `ping` uses ICMP, which firewalls often block even when real traffic works. A failed ping is a hint, not proof. A successful ping proves less than you think, because it says nothing about the port your application uses.

## Refused, timed out, unreachable

These three results mean different things, and recognizing them saves hours.

| Result | What happened | Likely cause |
|---|---|---|
| **Connection refused** | The host answered with a reset | The host is up but nothing listens on that port, or it was rejected explicitly |
| **Connection timed out** | No reply at all | A firewall or security group silently dropped the packets, or the route is wrong |
| **No route to host** / **Network unreachable** | The path is not known or an intermediate router reported failure | Routing or subnet configuration |

A firewall that **drops** packets produces a timeout. A firewall that **rejects** them, or a closed port, produces a quick refusal. So *fast failure* usually means "reachable but closed" and *slow failure* usually means "blocked".

## Testing a port

```
$ nc -zv -w 3 10.0.2.15 5432
nc: connect to 10.0.2.15 port 5432 (tcp) failed: Connection timed out
$ nc -zv -w 3 10.0.2.15 22
Connection to 10.0.2.15 22 port [tcp/ssh] succeeded!
```

`-z` scans without sending data, `-v` is verbose and `-w 3` times out after three seconds. This is sample output. Here SSH works but PostgreSQL does not, and the failure is a timeout. If `nc` is missing, Bash can test too: `timeout 3 bash -c '</dev/tcp/10.0.2.15/5432' && echo open`.

MARGA by EknathaLabs marga.eknathalabs.com

On the **server**, confirm what is actually listening:

```
$ sudo ss -tlnp | grep 5432
LISTEN 0 244 10.0.2.15:5432 0.0.0.0:* users:(("postgres",pid=845,fd=6))
```

The database listens on `10.0.2.15:5432`, so it is not the problem. The `Local Address` column matters: a service bound to `127.0.0.1` cannot be reached from another machine whatever the firewall says.

## Watch the packets

When tools disagree, look at the wire. `tcpdump` on the **server** shows whether packets arrive.

```
$ sudo tcpdump -ni eth0 'tcp port 5432 and host 10.0.1.12' -c 4
10:41:07.118 IP 10.0.1.12.51522 > 10.0.2.15.5432: Flags [S], seq 2812554981, win 64240, length 0
10:41:08.121 IP 10.0.1.12.51522 > 10.0.2.15.5432: Flags [S], seq 2812554981, win 64240, length 0
10:41:10.125 IP 10.0.1.12.51522 > 10.0.2.15.5432: Flags [S], seq 2812554981, win 64240, length 0
```

`[S]` is a SYN, the first packet of a TCP connection. The client keeps retrying and the server never answers with `[S.]` (SYN-ACK). If the packets show up on the server but get no reply, the **host firewall** is dropping them. If they never show up on the server at all, something *between* the machines (a network firewall, security group or routing rule) is blocking them.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How A layered troubleshooting toolkit fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Work from the bottom up** | Test each layer and stop at the first failure. | Treating a failed ping as proof the host is down. |
| **Refused, timed out, unreachable** | These three results mean different things, and recognizing them saves hours. | Debugging the application before checking the network path. |
| **Testing a port** | -z scans without sending data, -v is verbose and -w 3 times out after three seconds. | Testing from your laptop instead of from the machine that has the problem. |
| **Watch the packets** | When tools disagree, look at the wire. tcpdump on the server shows whether packets arrive. | The client keeps retrying and the server never answers with [S.] (SYN-ACK). |

Flow: Work from the bottom up to Refused, timed out, unreachable / Refused, timed out, unreachable to Testing a port / Testing a port to Watch the packets

## Incident: app cannot reach the database after a network change

> **Incident:** *After the platform team moved the database to a new subnet, the orders app logs* `connection timed out` *to* `10.0.2.15:5432`*.*

Apply the layers:

1. **Route:** `ip route get 10.0.2.15` shows a path through the gateway, so routing exists.
2. **Name:** the app uses an IP, so DNS is not involved.
3. **Port:** `nc -zv 10.0.2.15 5432` times out. A timeout, not a refusal, suggests packets are dropped.
4. **Server side:** `ss -tlnp` shows PostgreSQL listening on `10.0.2.15:5432`, so the service is healthy.
5. **Wire:** `tcpdump` on the database host shows **no SYN packets** arriving.

The packets never reach the server, so the block is between the subnets. The new database subnet's firewall rule allowed only the old application subnet. Add an allow rule from `10.0.1.0/24` to port 5432, then retest:

```
$ nc -zv -w 3 10.0.2.15 5432
Connection to 10.0.2.15 5432 port [tcp/postgresql] succeeded!
```

MARGA by EknathaLabs marga.eknathalabs.com

You did not need to touch the application or restart anything. Writing down the evidence from each layer also gives the network team exactly what they need, so the conversation takes minutes rather than days.

## Common mistakes

- Treating a failed ping as proof the host is down
- Debugging the application before checking the network path
- Forgetting that a service listening on `127.0.0.1` is unreachable from other hosts
- Testing from your laptop instead of from the machine that has the problem

### Check yourself

**A TCP connection attempt fails immediately with "connection refused". What is the most likely cause?**

Answer: The host is reachable but nothing is listening on that port

**nc -zv to a database port times out, while SSH to the same host works. What does this suggest?**

Answer: Traffic to that port is being dropped somewhere, such as a firewall or security group

**On the server, tcpdump shows repeated SYN packets from the client and no replies. Where is the block most likely?**

Answer: On the server itself, such as its host firewall, because the packets arrive but are not answered

**A service listens on 127.0.0.1:5432. Can another machine connect to it?**

Answer: No, it only accepts connections from the same machine

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 6. Firewalls and load balancing

16 minutes. Write safe firewall rules, understand security groups, and diagnose load balancers that mark healthy servers as down.

Two jobs share one lesson because they sit in the same place: in front of your servers, deciding which traffic gets through and where it goes. A firewall rule that is one line out of order can lock you out of a machine. A load balancer health check that expects the wrong response can take a healthy service out of rotation. Both are easy to reason about once you see how they decide.

## What you will learn

- Stateful versus stateless filtering
- A safe order for host firewall rules
- How cloud security groups differ from network ACLs
- How load balancers choose backends and check their health

## Firewall basics

A firewall matches packets against rules and applies an action: accept, drop (silently discard) or reject (discard and tell the sender). Two ideas matter most.

- **Stateful** firewalls track connections. If you allow an outbound request, the reply is allowed automatically. Most host firewalls and cloud security groups work this way.
- **Stateless** filters evaluate each packet alone, so you must allow both directions explicitly. Cloud network ACLs behave like this.

Use **default deny**: block everything, then allow only what you need.

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT   # replies and relate \
    d traffic
sudo iptables -A INPUT -i lo -j ACCEPT                                         # local loopback
sudo iptables -A INPUT -p tcp --dport 22  -j ACCEPT                            # SSH
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT                            # HTTPS
sudo iptables -P INPUT DROP                                                    # set default deny  \
    LAST
```

**Order matters.** Rules are checked from the top and the first match wins. The default policy `DROP` goes last so that you never close the door before you have allowed yourself back in. On Ubuntu, `ufw` expresses the same ideas more simply:

```
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

> **Warning:** *When you change firewall rules over SSH, schedule an automatic rollback first, for example* `echo 'sudo iptables-restore < /root/rules.good' | at now + 5 minutes`*, and cancel it once you have confirmed you can still log in.*

## Cloud equivalents

| Concept | AWS | Azure |
|---|---|---|
| Instance-level, stateful rules | Security group | Network security group (NSG) on NIC or subnet |
| Subnet-level rules | Network ACL (stateless) | NSG attached to a subnet (stateful) |
| Rule evaluation | Security group: all allow rules together. NACL: numbered, lowest first | NSG: priority number, lowest first |

A common pattern is to reference **groups instead of addresses**: allow the database's port from "the app security group" rather than from an IP range. New app instances then get access automatically.

MARGA by EknathaLabs marga.eknathalabs.com

## Load balancers

A **load balancer** spreads requests over several backends and removes failed ones.

| Type | Works on | Good for |
|---|---|---|
| Layer 4 (TCP/UDP) | Addresses and ports | Raw speed, non-HTTP protocols |
| Layer 7 (HTTP) | Paths, headers, cookies | Routing `/api` and `/static` separately, TLS termination |

Common algorithms are round robin, least connections and hashing on the client address. **Health checks** are central: the balancer probes each backend and sends traffic only to those that pass.

A basic nginx reverse proxy shows the idea:

```
upstream shop_api {
    least_conn;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=10s;
}

server {
    listen 443 ssl;
    server_name shop.example.com;

    location / {
        proxy_pass http://shop_api;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

Because the balancer terminates TLS and forwards plain HTTP, backends see the balancer's address. `X-Forwarded-For` carries the original client address so logs and rate limits stay meaningful.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How Firewalls and load balancing fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Firewall basics** | A firewall matches packets against rules and applies an action: accept, drop (silently discard) or reject (discard and tell the sender). | Setting the default policy to DROP before allowing SSH. |
| **Cloud equivalents** | A common pattern is to reference groups instead of addresses: allow the database's port from "the app security group" rather than from an IP range. | Allowing access by broad address ranges instead of by group. |
| **Load balancers** | A load balancer spreads requests over several backends and removes failed ones. | Health checks that follow redirects or require authentication. |

Flow: Firewall basics to Cloud equivalents / Cloud equivalents to Load balancers

## Incident: all backends marked unhealthy after a release

> **Incident:** *The load balancer reports 0 healthy targets and returns 503 to users. The application is running on both servers and serves real traffic directly.*

Test the health check the way the balancer does it:

```
$ curl -I http://10.0.1.11:8080/health
HTTP/1.1 301 Moved Permanently
Location: https://10.0.1.11:8080/health
```

The release added a rule redirecting all HTTP to HTTPS. The balancer's health check expects `200 OK` and treats the `301` as a failure, so it marked every backend down at once. The application itself was fine.

MARGA by EknathaLabs marga.eknathalabs.com

Choose one of two fixes: configure the health check to use a path excluded from the redirect, or let the backend answer `/health` over plain HTTP. Then confirm:

```
$ curl -s -o /dev/null -w '%{http_code}\n' http://10.0.1.11:8080/health
200
```

Two lessons carry over. Health checks should test **what the balancer can actually reach** with a simple, cheap endpoint, and they should **fail on real problems only**. A health endpoint that checks every dependency can take the whole fleet down when one dependency slows, so decide what "healthy" means on purpose.

## Predict the rule match

## Common mistakes

- Setting the default policy to DROP before allowing SSH
- Allowing access by broad address ranges instead of by group
- Health checks that follow redirects or require authentication
- Losing the client IP because `X-Forwarded-For` is not passed through

### Check yourself

**Why should the default DROP policy be set last when building iptables rules over SSH?**

Answer: Setting it first would cut your own session before the allow rules exist

**What is the main difference between a stateful and a stateless firewall?**

Answer: A stateful firewall tracks connections so replies are allowed automatically, while a stateless one needs rules for both directions

**A health check receives 301 from every backend and marks them unhealthy. What is the best fix?**

Answer: Point the check at a path that returns 200 without redirecting

**What does X-Forwarded-For carry?**

Answer: The original client's address, preserved by the proxy

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 7. VPNs, tunnels and private connectivity

15 minutes. Connect networks securely with WireGuard and IPsec, use SSH and TLS tunnels, and understand cloud site-to-site VPN, peering and private endpoints.

Sooner or later two private networks must talk to each other: an office and a cloud network, two cloud VPCs, or a laptop and a production database. A **tunnel** carries private traffic across an untrusted network, usually by encrypting and encapsulating packets.

## What you will learn

- What a VPN actually does at the packet level
- A WireGuard site-to-site example
- IPsec and how cloud VPN gateways use it
- Alternatives: peering, private endpoints and SSH tunnels

## How a tunnel works

The original packet is encrypted, wrapped inside a new packet addressed to the VPN peer, and sent across the internet. The peer unwraps and decrypts it, then delivers the original into its own network. To both sides, the networks look directly connected.

Key design questions are the same every time: which address ranges live on each side, which side initiates, how the keys are exchanged, and what happens when the tunnel drops.

## WireGuard

WireGuard is small, fast and simple. Each peer has a key pair, and configuration lists which peers may send which address ranges.

```
$ umask 077
$ wg genkey | tee private.key | wg pubkey > public.key
$ sudo wg show
```

```
[Interface]
Address=10.99.0.1/24
ListenPort=51820
PrivateKey=REPLACE_WITH_PRIVATE_KEY

[Peer]
PublicKey=REPLACE_WITH_PEER_PUBLIC_KEY
AllowedIPs=10.99.0.2/32,192.168.50.0/24
Endpoint=203.0.113.20:51820
PersistentKeepalive=25
```

Save that as `/etc/wireguard/wg0.conf`, then bring it up:

```
$ sudo wg-quick up wg0
$ sudo systemctl enable wg-quick@wg0
$ ping -c 2 10.99.0.2
$ sudo wg show wg0 latest-handshakes
```

`AllowedIPs` does two jobs: it decides what traffic is routed into the tunnel, and it filters which source addresses the peer may use. A recent handshake time proves keys match and UDP is getting through. Allow UDP 51820 in firewalls and security groups.

## IPsec and cloud VPNs

IPsec is the long-standing standard supported by nearly every router and firewall. Clouds expose it as a **site-to-site VPN**: a virtual private gateway in your VPC (AWS) or VPN gateway in a VNet (Azure) builds two redundant tunnels to your on-premises device. You configure the pre-shared key or certificate, the IKE and IPsec encryption parameters, and either static routes or BGP.

MARGA by EknathaLabs
marga.eknathalabs.com

Both ends must agree exactly on parameters, and the usual failures are a mismatched proposal, a wrong pre-shared key, overlapping CIDRs or a missing route on one side. Use both tunnels for availability, and monitor them: a tunnel that is down silently is a common source of "intermittent" problems.

## Private connectivity inside the cloud

Not every connection needs a VPN.

| Need | Tool |
|---|---|
| Two VPCs or VNets in one cloud | VPC peering, VNet peering or a transit hub |
| Reach a cloud service without the internet | Private endpoint (PrivateLink, Private Link) |
| Dedicated, predictable bandwidth from a data centre | Direct Connect, ExpressRoute |
| Quick access to one internal service | SSH or SSM port forwarding |

Peering is not transitive: if A peers with B and B with C, A cannot reach C through B. A **transit gateway** or hub-and-spoke design provides routing between many networks.

```
$ ssh -N -L 15432:db.internal:5432 ops@bastion
$ psql -h 127.0.0.1 -p 15432 -U app appdb
$ aws ssm start-session --target i-0abc123def --document-name AWS-StartPortForwardingSession --par \
    ameters portNumber=5432,localPortNumber=15432
```

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How VPNs, tunnels and private connectivity fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **How a tunnel works** | The original packet is encrypted, wrapped inside a new packet addressed to the VPN peer, and sent across the internet. | Overlapping address ranges on the two sides. |
| **WireGuard** | WireGuard is small, fast and simple. Each peer has a key pair, and configuration lists which peers may send which address ranges. | Forgetting routes or security rules after the tunnel is up. |
| **IPsec and cloud VPNs** | IPsec is the long-standing standard supported by nearly every router and firewall. | A single tunnel with no failover and no monitoring. |
| **Private connectivity inside the** | Not every connection needs a VPN. | Treating peering as transitive. |

Flow: How a tunnel works to WireGuard / WireGuard to IPsec and cloud VPNs / IPsec and cloud VPNs to Private connectivity inside the

## Incident: the tunnel that was up but carried nothing

> **Incident:** *The site-to-site VPN shows both tunnels as UP in the console, yet the application in the cloud cannot reach the on-premises database. Ping works from the gateway, but not from application servers.*

The tunnel is a transport; routing still decides which traffic enters it. Check each layer:

```
$ ip route get 192.168.50.10
192.168.50.10 via 10.0.0.1 dev eth0 src 10.0.3.22
$ traceroute -n 192.168.50.10
```

The cloud route table for the application subnet had no route for 192.168.50.0/24 via the VPN gateway, so traffic went to the internet gateway and was dropped. Fix the route table (or enable route propagation), then verify the on-premises firewall also allows the database port from the cloud CIDR.

## Common mistakes

- Overlapping address ranges on the two sides

MARGA by EknathaLabs marga.eknathalabs.com

- Forgetting routes or security rules after the tunnel is up
- Treating peering as transitive
- A single tunnel with no failover and no monitoring
- Sharing pre-shared keys in chat or tickets

## Check yourself

**What does a VPN tunnel do?**

Answer: Encrypts and encapsulates private traffic so it can cross an untrusted network

**What are AllowedIPs in WireGuard used for?**

Answer: Routing traffic into the tunnel and restricting which sources a peer may use

**Are VPC peering connections transitive?**

Answer: No, A to B and B to C does not give A access to C

**A VPN tunnel is up but traffic does not flow. What do you check first?**

Answer: Route tables and security rules on both sides

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 8. IPv6 essentials for engineers

14 minutes. Read and write IPv6 addresses, understand prefixes, SLAAC, neighbour discovery and dual-stack operation, and avoid the common mistakes in firewalls, DNS and cloud networks.

IPv4 addresses ran out years ago. Mobile networks, large cloud providers and many ISPs now carry a large share of traffic over IPv6, and some environments are IPv6-only. You do not need to become an expert, but you must be able to read an address, know what a /64 is, and notice when something works over IPv4 but breaks over IPv6.

## What you will learn

- The structure of an IPv6 address and how to shorten it
- Address types: global, unique local, link-local, multicast
- How hosts get addresses: SLAAC, DHCPv6 and static
- Neighbour discovery instead of ARP
- Dual stack and the mistakes it hides

## Reading addresses

An IPv6 address is 128 bits written as eight groups of four hexadecimal digits: `2001:0db8:0000:0000:0000:0000:0000:0001`. Two rules shorten it: drop leading zeros in a group, and replace one run of all-zero groups with `::`. That gives `2001:db8::1`. The `::` may appear only once.

| Prefix | Meaning |
|---|---|
| `2000::/3` | Global unicast, routable on the internet |
| `fd00::/8` | Unique local, like private IPv4 ranges |
| `fe80::/10` | Link-local, valid only on one network segment |
| `ff00::/8` | Multicast |
| `::1/128` | Loopback |

A normal subnet is a `/64`: 64 bits of network and 64 bits of host. A site is often given a `/48` or `/56`, which holds 65,536 or 256 subnets. Do not try to save addresses by using smaller subnets; many features, including SLAAC, assume `/64`.

## How hosts get an address

- **SLAAC** (stateless autoconfiguration): the router announces the prefix in a router advertisement, and the host builds its own address from it. No server keeps a lease.
- **DHCPv6**: a server hands out addresses and options, closer to IPv4 behaviour.
- **Static**: set by hand or by automation for servers.

Every interface also has a link-local `fe80::` address, which is used for neighbour discovery and as the next hop in routing.

```
$ ip -6 addr show dev eth0
inet6 2001:db8:10:1::25/64 scope global
inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
$ ip -6 route
default via fe80::1 dev eth0
$ ping -6 -c1 2001:db8:10:1::1
$ curl -6 -sv https://example.com/ -o /dev/null
```

## Neighbour discovery

IPv6 has no ARP. Hosts use ICMPv6 messages (neighbour solicitation and advertisement, router solicitation and advertisement) to find each other and routers. This means you must not block ICMPv6 wholesale: if you do, hosts cannot find their router or neighbours, and path MTU discovery fails.

MARGA by EknathaLabs
marga.eknathalabs.com

## Dual stack

Most networks run IPv4 and IPv6 side by side. A client with both prefers IPv6 (and tries IPv4 quickly if it fails, "happy eyeballs"). Common mistakes:

- An AAAA DNS record exists but the server or firewall does not accept IPv6 connections.
- Security groups or firewall rules were written only for IPv4.
- Logs and allow lists store only IPv4 addresses, and IPv6 clients appear as something else or fail validation.
- Applications compare or parse addresses as text and break on colons.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How IPv6 essentials for engineers fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Reading addresses** | An IPv6 address is 128 bits written as eight groups of four hexadecimal digits: 2001:0db8:0000:0000:0000:0000:0000:0001. | Using subnets smaller than /64. |
| **How hosts get an address** | SLAAC (stateless autoconfiguration): the router announces the prefix in a router advertisement, and the host builds its own address from it. | Logging or validating addresses as IPv4 only. |
| **Neighbour discovery** | IPv6 has no ARP. Hosts use ICMPv6 messages (neighbour solicitation and advertisement, router solicitation and advertisement) to find each other and routers. | Blocking all ICMPv6. |
| **Dual stack** | Most networks run IPv4 and IPv6 side by side. | IPv6 records without IPv6 firewall rules. |

Flow: Reading addresses to How hosts get an address / How hosts get an address to Neighbour discovery / Neighbour discovery to Dual stack

## Incident: slow site for some users

**Incident:** *A site loads in two seconds for most users and takes more than thirty seconds for users on one mobile network.*

```
$ dig +short AAAA shop.example.com
2001:db8:50::10
$ curl -6 -m 10 -sv https://shop.example.com/ -o /dev/null
* connect to 2001:db8:50::10 port 443 failed: Connection timed out
```

The team had added an AAAA record during a migration, but the new load balancer's firewall allowed only IPv4. Clients on IPv6-first networks waited for the connection to time out before falling back. The fix was to open IPv6 on the firewall, test with `curl -6` in the deploy checklist, and monitor from IPv6-only vantage points.

## Common mistakes

- Blocking all ICMPv6
- IPv6 records without IPv6 firewall rules
- Using subnets smaller than /64
- Logging or validating addresses as IPv4 only

### Check yourself

**What is the usual size of an IPv6 subnet?**

Answer: /64

MARGA by EknathaLabs
marga.eknathalabs.com

**Which prefix is link-local?**

Answer: fe80

**Why should you not block ICMPv6 completely?**

Answer: Neighbour discovery and path MTU discovery depend on it

**A site is slow only for IPv6-first users after adding an AAAA record. What is the likely cause?**

Answer: The IPv6 path is blocked or not served, so clients wait before falling back

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 9. Latency, bandwidth, MTU and network performance

14 minutes. Separate latency from bandwidth, understand TCP behaviour on slow or lossy links, find MTU problems and measure throughput with the right tools.

"The network is slow" can mean a long round trip, a small pipe, packet loss, or a packet size problem. Each looks different and has a different fix. Learning to separate them saves days of guessing, especially for traffic between regions, through VPNs and across tunnels.

## What you will learn

- The difference between latency, bandwidth and throughput
- Why a high round trip time limits a single TCP connection
- Packet loss and its effect
- MTU, fragmentation and "black hole" connections
- Tools: `ping`, `mtr`, `iperf3`, `ss -ti`, `curl` timings

## Latency and bandwidth

**Latency** is how long a packet takes; **round trip time (RTT)** is there and back. Light in fibre travels roughly 200 km per millisecond, so London to Singapore cannot be below about 85 ms RTT, whatever you pay for. **Bandwidth** is how much can be sent per second. A pipe can be wide and still slow to answer, like a motorway to a distant city.

A TCP connection can have only a window of data in flight at once. Throughput is roughly `window / RTT`. With a 64 KB window and 100 ms RTT, one connection reaches about 5 Mbit/s, even on a gigabit link. Modern systems scale windows automatically, but loss and small buffers still limit long-distance transfers. Remedies: multiple parallel connections, bigger buffers, a CDN or a closer endpoint, and protocols that need fewer round trips.

## Round trips add up

Opening a secure web request over a new connection costs a TCP handshake, a TLS handshake and then the request, several round trips before the first byte. At 150 ms RTT that is nearly half a second. Connection reuse (keep-alive, HTTP/2), TLS session resumption and HTTP/3 reduce this. Count round trips before you buy bandwidth.

## Loss

Even 1 percent packet loss can cut TCP throughput a lot, because the sender slows down at each loss. Find where loss starts:

```
$ mtr -rwzc 100 api.example.com
HOST                Loss%  Snt  Last  Avg  Best  Wrst
1. gw.local          0.0%  100   1.0  1.1   0.9   2.3
4. isp-core-2        0.0%  100  12.1 12.4  11.8  20.1
7. transit-9        18.0%  100  88.0 90.2  85.0 210.5
9. api.example.com   0.0%  100  91.0 91.3  90.1  99.7
```

Loss on a middle hop that does not continue to the destination (hop 7 above, but hop 9 is clean) is usually routers limiting their own replies, not real loss. Loss that continues to the last hop is real.

## MTU

The maximum transmission unit is the largest packet a link carries, usually 1500 bytes. Tunnels, VPNs and some cloud networks add headers, so the usable size is smaller (for example 1420 for WireGuard, or 8900 on a jumbo-frame path). If a packet is too large and cannot be fragmented, a router should send back an ICMP message "fragmentation needed"; if that message is blocked, the connection hangs when large packets are sent: small requests work, large responses stall. This is an MTU black hole.

MARGA by EknathaLabs marga.eknathalabs.com

```
ping -M do -s 1472 -c 3 host    # 1472 + 28 header bytes = 1500; fails if the path is smaller
ip link show dev wg0            # see the interface MTU
ss -ti dst host                 # see rtt, cwnd, retransmits for live connections
iperf3 -c server -P 4 -t 10     # measure throughput with 4 parallel streams
curl -o /dev/null -sw 'dns %{time_namelookup} connect %{time_connect} tls %{time_appconnect} first \
     byte %{time_starttransfer}\n' https://api.example.com/
```

The fix for MTU trouble is to allow the ICMP messages, lower the MTU on the tunnel, or clamp the TCP maximum segment size on the router.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How Latency, bandwidth, MTU and network performance fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Latency and bandwidth** | Latency is how long a packet takes; round trip time (RTT) is there and back. | Testing only with small pings. |
| **Round trips add up** | Opening a secure web request over a new connection costs a TCP handshake, a TLS handshake and then the request, several round trips before the first byte. | Buying bandwidth to fix a latency problem. |
| **Loss** | Even 1 percent packet loss can cut TCP throughput a lot, because the sender slows down at each loss. | Reading loss on a middle hop as the cause. |
| **MTU** | The maximum transmission unit is the largest packet a link carries, usually 1500 bytes. | Blocking all ICMP. |

Flow: Latency and bandwidth to Round trips add up / Round trips add up to Loss / Loss to MTU

## Incident: the VPN that only worked for small requests

> **Incident:** *After a new site-to-site VPN is set up, health checks pass but file downloads between offices stall at around 1 KB.*

```
$ ping -M do -s 1472 -c 2 10.20.0.5
ping: local error: message too long, mtu=1400
```

The tunnel's MTU was 1400 and a firewall dropped the ICMP messages that tell senders to use smaller packets. The team set the MTU on both ends to 1400, clamped the TCP MSS to 1360, allowed the required ICMP types, and added a large-packet test to the VPN checklist.

## Common mistakes

- Buying bandwidth to fix a latency problem
- Testing only with small pings
- Reading loss on a middle hop as the cause
- Blocking all ICMP

### Check yourself

**A transfer between two regions with 150 ms RTT is slow on one connection although the link is fast. What often limits it?**

Answer: The amount of data that can be in flight per round trip

**What is an MTU black hole?**

Answer: Large packets are dropped silently because needed ICMP messages are blocked

**Which command checks whether a 1500-byte packet fits the path without fragmentation?**

Answer: ping -M do -s 1472 host

MARGA by EknathaLabs marga.eknathalabs.com

**In mtr, loss at a middle hop that disappears at the final hop usually means:**

Answer: That router limits replies to probes and the path is not really lossy

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 10. Segmentation, security groups and least-privilege networks

13 minutes. Divide a network into zones, write firewall and security group rules that allow only what is needed, and review them.

A flat network is easy to build and easy to lose: once an attacker reaches one machine, every other machine is one hop away. Segmentation limits that reach. In a cloud, the same idea appears as subnets, security groups and network ACLs, and a few habits keep the rules understandable.

## What you will learn

- Zones such as public, application and data, and why they exist
- How security groups and network ACLs differ
- Writing rules by purpose: source, port, reason
- Reviewing and testing rules

## Zones and the path of a request

A typical design has three tiers. The **public** tier holds load balancers that accept traffic from the internet. The **application** tier runs services that only the load balancer may call. The **data** tier holds databases that only the application may reach. Each tier sits in its own subnet, and each boundary is a place where you decide what may cross.

```
internet -> load balancer (443) -> app servers (8080) -> database (5432)
```

If the web tier is compromised, the attacker still faces rules between it and the database. Add a separate **management** zone for bastion hosts and tooling, with its own rules.

## Security groups and ACLs

In most clouds there are two filters. A **security group** attaches to an instance or interface. It is *stateful*: if a connection is allowed in, the reply is allowed out automatically. Rules are allow-only, and anything not allowed is denied. A **network ACL** belongs to a subnet, is usually *stateless* (return traffic needs its own rule) and can allow or deny.

Use security groups as the main tool. A powerful habit is to reference another group as the source, not an address range:

```
SG db:   allow TCP 5432 from SG app
SG app:  allow TCP 8080 from SG lb
SG lb:   allow TCP 443  from 0.0.0.0/0
```

Now the rules follow the machines as they scale, and a new app server is allowed to reach the database without anyone editing an address list.

## Write rules with a reason

Every rule should answer: who, to what, on which port, and why. A rule such as "allow all from 10.0.0.0/8" answers none of those questions. Name rules and add a description that links to a ticket or a service.

- Open only the ports a service needs; avoid ranges such as 0-65535.
- Never expose administration ports (SSH 22, RDP 3389, databases) to 0.0.0.0/0. Use a bastion, a VPN or a managed session service.
- Limit outbound traffic from the data tier. Databases rarely need to reach the internet.
- Use tags and group names that describe the role, not the person.

MARGA by EknathaLabs
marga.eknathalabs.com

## Check what is really allowed

Rules drift. Review them on a schedule and test the paths that should be blocked, not only the ones that should work.

```
$ nc -zv -w3 db.internal 5432      # from an app host: should succeed
$ nc -zv -w3 db.internal 5432      # from a web host: should time out
$ aws ec2 describe-security-groups --query "SecurityGroups[?IpPermissions[?IpRanges[?CidrIp=='0.0. \
    0.0/0']]].GroupId"
```

The last command lists security groups that contain an open-to-the-world rule. Cloud tools can also record network flows (flow logs), which shows traffic that was rejected and helps you find rules that are missing or too wide.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How Segmentation, security groups and least-privilege... fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Zones and the path of a request** | A typical design has three tiers. The public tier holds load balancers that accept traffic from the internet. | Rules without descriptions or owners. |
| **Security groups and ACLs** | In most clouds there are two filters. A security group attaches to an instance or interface. | Testing only the allowed paths, never the blocked ones. |
| **Write rules with a reason** | Every rule should answer: who, to what, on which port, and why. | Opening 0.0.0.0/0 for administration ports. |
| **Check what is really allowed** | Rules drift. Review them on a schedule and test the paths that should be blocked, not only the ones that should work. | Allowing a whole private range because it is convenient. |

Flow: Zones and the path of a request to Security groups and ACLs / Security groups and ACLs to Write rules with a reason / Write rules with a reason to Check what is really allowed

## Incident: Everything could reach the database

**Incident:** *A pen test found that a forgotten test server in the public subnet could connect to the production database on port 5432.*

```
$ aws ec2 describe-security-groups --group-ids sg-0db1 --query "SecurityGroups[0].IpPermissions"
[ { "FromPort": 5432, "IpRanges": [ { "CidrIp": "10.0.0.0/8", "Description": "temporary" } ] } ]
```

The database group allowed the whole private range "temporarily" two years earlier. Any machine in the network could connect, including the test server. The fix replaced it with a rule that allows only the application group, flow logs were enabled on the data subnet, and a monthly review lists rules with the word temporary or without a description.

## Common mistakes

- Allowing a whole private range because it is convenient
- Opening 0.0.0.0/0 for administration ports
- Rules without descriptions or owners
- Testing only the allowed paths, never the blocked ones

### Check yourself

**What does stateful mean for a security group?**

Answer: Replies to allowed connections are allowed automatically

**Which design best protects a database?**

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Only the application tier may connect to it, on one port

**Why avoid 0.0.0.0/0 on port 22?**

Answer: It exposes administration to the whole internet

**What do flow logs help with?**

Answer: Seeing which connections were accepted or rejected, to find missing or too wide rules

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 11. DHCP, ARP and the local network

12 minutes. Understand how a host gets an address, finds neighbours on its own network, and what fails when either step breaks.

A laptop connects to the network and shows an address starting with 169.254. A server answers pings from some machines and not others, because two machines claim the same address. Both problems live on the local network, below DNS and routing, and both are quick to diagnose once you know the two small protocols involved.

## What you will learn

- How DHCP leases an address, a gateway and a DNS server
- What the 169.254 address means
- How ARP maps an IP address to a hardware address
- Spotting duplicate addresses and stale ARP entries

## DHCP in four steps

A new host has no address, so it asks for one by broadcast. The exchange is called DORA:

1. **Discover**: the client broadcasts "is there a DHCP server?".
2. **Offer**: a server proposes an address, a lease time, a gateway and DNS servers.
3. **Request**: the client asks for that address.
4. **Acknowledge**: the server confirms, and the lease starts.

Halfway through the lease the client tries to renew. If it never gets an answer, many systems assign themselves a **link-local** address in `169.254.0.0/16`. Seeing one means "DHCP did not answer", not "the network is broken everywhere".

```
$ ip -br addr
eth0   UP   10.0.3.27/24
$ journalctl -u systemd-networkd | grep -i dhcp | tail -3
$ sudo dhclient -v eth0
```

## Reservations and ranges

A DHCP server hands out addresses from a **pool**. Servers that others depend on should have either a static address outside the pool or a **reservation**, which always gives the same address to one hardware address. Mixing the two is where duplicates come from: someone types a static address that lies inside the pool, and later the server hands the same address to a laptop.

Keep a record of which ranges are static, which are dynamic and which are reserved. In the cloud, the platform acts as the DHCP server, and you set addresses in the virtual network configuration, not on the machine.

## ARP: who has this address?

On an Ethernet network, frames are delivered by hardware (MAC) address. To send to 10.0.3.1, a host broadcasts an **ARP** request, "who has 10.0.3.1?", and the owner replies with its MAC. The answer is kept in a cache for a short time.

```
$ ip neigh
10.0.3.1 dev eth0 lladdr 52:54:00:12:35:02 REACHABLE
10.0.3.40 dev eth0 FAILED
$ sudo arping -I eth0 10.0.3.27
```

`FAILED` means nobody answered, so the host is down, on another network, or blocked. ARP works only inside one broadcast domain. Traffic for other networks goes to the gateway, so you only ARP for the gateway's address.

MARGA by EknathaLabs marga.eknathalabs.com

## Duplicate addresses and stale entries

If two machines use one address, ARP replies conflict, and traffic flips between them. Symptoms are connections that work for a minute and then reset, and a MAC address that changes between checks.

```
$ sudo arping -D -I eth0 -c 2 10.0.3.27
$ ip neigh show 10.0.3.27
$ sudo ip neigh flush 10.0.3.27
```

`arping -D` does a duplicate address detection check, and flushing an entry forces a fresh lookup. After a failover that moves an address to another machine, a gratuitous ARP announces the change so neighbours update their caches quickly.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How DHCP, ARP and the local network fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **DHCP in four steps** | A new host has no address, so it asks for one by broadcast. | Typing a static address that lies inside the DHCP pool. |
| **Reservations and ranges** | A DHCP server hands out addresses from a pool. | Assuming a 169.254 address means a faulty machine. |
| **ARP: who has this address?** | On an Ethernet network, frames are delivered by hardware (MAC) address. | Check the Common mistakes list below this diagram for what usually goes wrong here. |
| **Duplicate addresses and stale** | If two machines use one address, ARP replies conflict, and traffic flips between them. | Forgetting that ARP does not cross routers. |

Flow: DHCP in four steps to Reservations and ranges / Reservations and ranges to ARP: who has this address? / ARP: who has this address? to Duplicate addresses and stale

## Incident: Two servers, one address

> **Incident:** *After a hardware swap, a monitoring dashboard flickered between healthy and failed for an internal API at 10.0.3.27.*

```
$ ip neigh show 10.0.3.27
10.0.3.27 dev eth0 lladdr 52:54:00:aa:01:10 REACHABLE
$ ip neigh show 10.0.3.27
10.0.3.27 dev eth0 lladdr 52:54:00:bb:02:20 REACHABLE
```

The replacement server had been given the old address by hand while the old machine was still powered on in a rack. The MAC address behind the IP kept changing. Powering off the old machine ended the flicker. The team then moved addresses for infrastructure into a documented static range outside the DHCP pool, and added duplicate address detection to the provisioning script.

## Common mistakes

- Typing a static address that lies inside the DHCP pool
- Assuming a 169.254 address means a faulty machine
- Forgetting that ARP does not cross routers
- Leaving old machines powered on after migration

### Check yourself

**What does a 169.254.x.x address usually indicate?**

Answer: The host did not get an answer from a DHCP server

**What does ARP do?**

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Finds the hardware address that belongs to an IP address on the local network

**What are the four DHCP messages?**

Answer: Discover, Offer, Request, Acknowledge

**Which symptom suggests a duplicate IP address?**

Answer: The MAC address for one IP changes between checks and connections reset

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 12. Debugging network problems layer by layer

15 minutes. Work up the stack from link to DNS, routing, ports and TLS with ping, dig, curl, ss and traceroute, so you can say exactly which layer is broken.

"The network is down" is rarely true. Usually one layer fails while the others work: a name does not resolve, a port is filtered, a certificate has expired, or a route is missing. Testing one layer at a time turns a vague complaint into a specific finding.

## What you will learn

- A bottom-up checklist: link, address, route, DNS, port, TLS, application
- What each of `ip`, `ping`, `dig`, `traceroute`, `ss`, `curl` and `tcpdump` proves
- How to separate "cannot resolve" from "cannot connect" from "connected but wrong answer"

## The ladder

| Layer | Question | Command |
|---|---|---|
| Link and address | Do I have an address? | `ip addr`, `ip link` |
| Route | Is there a path out? | `ip route`, `traceroute -n host` |
| Name | Does the name resolve? | `dig +short host`, `resolvectl query host` |
| Port | Can I reach the port? | `nc -vz host 443`, `curl -v telnet://host:443` |
| TLS | Is the certificate valid? | `openssl s_client -connect host:443 -servername host` |
| Application | Does it answer correctly? | `curl -sv https://host/health` |

## One request, step by step

```
$ dig +short api.example.com
203.0.113.25
$ nc -vz 203.0.113.25 443
Connection to 203.0.113.25 443 port [tcp/https] succeeded!
$ curl -sv https://api.example.com/health -o /dev/null 2>&1 | grep -E "subject|expire|HTTP/"
*  subject: CN=api.example.com
*  expire date: Oct  3 00:00:00 2026 GMT
< HTTP/2 503
```

The name resolves, the port opens, the certificate is valid, and the server answers 503. The network is healthy; the application behind it is not. Without the ladder you might have spent an hour on firewalls.

## Reading failures

- **Could not resolve host** means DNS. Check `/etc/resolv.conf`, try `dig @8.8.8.8`, and compare.
- **Connection timed out** usually means a firewall or security group silently drops packets, or no route exists.
- **Connection refused** means the host answered and nothing listens on that port.
- **Certificate errors** mean expiry, wrong name, or a missing intermediate certificate.

## Look at the packets

```
sudo tcpdump -ni any host 203.0.113.25 and port 443 -c 20
ss -tan state syn-sent        # connections stuck waiting for a reply
ss -ltnp                      # what is listening locally, and which process owns it
```

MARGA by EknathaLabs marga.eknathalabs.com

Many `SYN-SENT` entries with no reply point to filtering or a dead target. A server that shows `LISTEN` on `127.0.0.1:8080` only will refuse outside traffic even though the service is "up".

### Diagram: Debugging order for a failing connection

| Part | What it does | What goes wrong |
|---|---|---|
| **Address and link** | ip addr shows whether the host has an address and the interface is up. | No address means DHCP or interface trouble, and everything above fails. |
| **Route** | ip route and traceroute show the path toward the target. | A missing default route or wrong gateway sends packets nowhere. |
| **DNS** | dig turns the name into an address. | A wrong or stale answer sends clients to the wrong place. |
| **Port and firewall** | nc and ss show whether the port answers. | Security groups and filters drop packets, producing timeouts. |
| **TLS** | openssl s_client checks the certificate and name. | An expired or mismatched certificate fails after the connection opens. |
| **Application** | curl shows the real status code and body. | The network may be fine while the service returns errors. |

Flow: Address and link to Route / Route to DNS / Route to Port and firewall / DNS to TLS / Port and firewall to TLS / TLS to Application

## Incident: DNS that worked for some

**Incident:** *After a migration, half of the users reach the new service and half still land on the old one.*

```
$ dig +short shop.example.com @ns1.example.net
203.0.113.40
$ dig +short shop.example.com @8.8.8.8
198.51.100.9
$ dig shop.example.com | grep -A1 "ANSWER SECTION"
shop.example.com.   86400  IN  A  198.51.100.9
```

The authoritative server had the new address, but the record carried a 24-hour TTL, so resolvers kept serving the old one. The team lowered the TTL a day before the next migration, and verified with both authoritative and public resolvers.

## Common mistakes

- Testing only with `ping`, which may be blocked while the service works
- Forgetting that the service might listen on localhost only
- Ignoring DNS caches and TTL
- Changing firewall rules before proving which layer fails

### Check yourself

**"Connection refused" tells you which of these?**

Answer: The host answered, but nothing accepts connections on that port

**Which command shows what is listening on local ports and which process owns them?**

Answer: ss -ltnp

**Users report that some reach the old service after a DNS change. What is the most likely cause?**

Answer: Resolvers are caching the old record until its TTL expires

**A curl to a host times out after 30 seconds. What is the most useful next step?**

Answer: Check routes and firewall or security group rules, and test with nc or tcpdump

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 13. DNS, TLS and PKI: DNS records, resolution and caching

13 minutes. Understand how a name becomes an address through recursive and authoritative servers, what each record type is for, and how TTLs and caches decide how fast changes spread.

Almost every outage story has a DNS chapter. The system is simple in principle (names map to data), but it is spread across caches you do not control. Knowing who answers, who caches and for how long turns "DNS is weird" into a predictable picture.

## What you will learn

- The roles of stub, recursive and authoritative servers
- The record types you meet daily
- How TTL, caching and negative caching shape change speed
- What delegation and the apex CNAME rule mean in practice

## Who answers

A program asks a **stub resolver** on the host, which asks a **recursive resolver** (your provider, the cloud VPC resolver or a company server). The recursive resolver walks from the root servers to the TLD servers to the domain's **authoritative** servers, caches what it learns, and returns the answer. Only the authoritative servers hold the truth; everyone else holds copies that expire.

```
$ dig +trace www.example.com
$ dig NS example.com +short
$ dig @ns1.example.com www.example.com
```

`+trace` shows the walk. Asking an authoritative server directly with `@` shows the real data without any cache in between, which is the first test when you suspect stale answers.

## Record types

- **A** and **AAAA**: IPv4 and IPv6 addresses
- **CNAME**: an alias to another name; the target must be resolved too
- **MX**: mail servers with priority
- **TXT**: free text, used for SPF, domain verification and ACME challenges
- **NS**: the authoritative servers for a zone (delegation)
- **SOA**: zone metadata, including the negative-caching time
- **SRV** and **PTR**: service location and reverse lookups

A name that has a CNAME cannot hold other records, so the zone **apex** (example.com itself) cannot be a plain CNAME, because it must also have NS and SOA. Providers solve this with ALIAS or ANAME features that flatten the answer on their side.

## TTL and caching

Each record carries a **TTL** in seconds. Resolvers may keep the answer that long. A TTL of 3600 means a change can take up to an hour to reach every cache, and you cannot reach into other people's caches to clear them. Failed lookups are cached too: an `NXDOMAIN` answer is kept for the SOA's negative TTL, so creating a record right after a failed lookup may still look missing for a while. Lower the TTL **before** a planned change (a day ahead, to the old TTL's length), make the change, then raise it again.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: How DNS, TLS and PKI: DNS records, resolution and... fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Who answers** | A program asks a stub resolver on the host, which asks a recursive resolver (your provider, the cloud VPC resolver or a company server). | Changing records without lowering the TTL first. |
| **Record types** | A and AAAA: IPv4 and IPv6 addresses. | Putting a CNAME at the zone apex. |
| **TTL and caching** | Each record carries a TTL in seconds. Resolvers may keep the answer that long. | Forgetting that failed lookups are cached too. |

Flow: Who answers to Record types / Record types to TTL and caching

## Incident: The migration that took a day to finish

> **Incident:** *A team moved a website to a new load balancer and changed the DNS record, but half of the customers kept reaching the old, switched-off address for most of a day.*

```
$ dig +noall +answer www.shop.example
www.shop.example. 85012 IN A 203.0.113.10
$ dig @ns1.shop.example +noall +answer www.shop.example
www.shop.example. 86400 IN A 198.51.100.20
```

The record had a TTL of 86400 seconds. A public resolver still returned the old address with 85012 seconds left, while the authoritative server already had the new one. The team switched the old address back on as a reverse proxy to the new site while caches expired, and for later moves they lowered TTLs to 300 seconds a day ahead and added a checklist item to compare answers from the authoritative server and two public resolvers.

## Common mistakes

- Changing records without lowering the TTL first
- Testing only through a caching resolver
- Putting a CNAME at the zone apex
- Forgetting that failed lookups are cached too

### Check yourself

**Which servers hold the authoritative data for a zone?**

Answer: The zone's authoritative name servers

**Why can the zone apex not be a plain CNAME?**

Answer: It must also hold NS and SOA records, and a CNAME allows no other records

**What does a TTL control?**

Answer: How long a resolver may keep the answer in its cache

**What is the best first test when you suspect stale DNS answers?**

Answer: Query the authoritative server directly with dig @server

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 14. DNS, TLS and PKI: debugging DNS failures

12 minutes. Tell NXDOMAIN, SERVFAIL and timeouts apart, test each resolver in the path, and find search-domain and Kubernetes ndots problems that multiply lookups.

When "the app cannot connect", the name lookup is the first thing to prove or rule out. A few dig commands tell you whether the name exists, which server is wrong, and whether the client is the cause.

## What you will learn

- Read the status in a DNS answer
- Test the stub, the recursive resolver and the authoritative server in turn
- Spot search domains, `ndots` and split-horizon effects
- Debug name resolution inside a Kubernetes pod

## Read the status

```
$ dig www.example.com | grep -E 'status|ANSWER'
$ dig www.example.com AAAA +short
$ dig @1.1.1.1 www.example.com +short
$ resolvectl query www.example.com
```

- `NOERROR` with an answer: the name resolved
- `NOERROR` with no answer (NODATA): the name exists but not this record type
- `NXDOMAIN`: the name does not exist (typo, missing record, wrong zone)
- `SERVFAIL`: the resolver could not get a valid answer, often a broken delegation, an unreachable authoritative server or a DNSSEC failure
- No reply (timeout): a network, firewall or dead-server problem

## Test each hop

1. The host's own view: `getent hosts name` uses the same libraries as applications, including `/etc/hosts` and search domains.
2. The configured resolver: `dig @ip name`.
3. A different public resolver: if it works there, your resolver is the issue.
4. The authoritative server: `dig @ns name`.

A `dig` that works while the application fails often means the application uses a different resolver path (a container, a VPN, a proxy or a cached negative result).

## Search domains and ndots

`/etc/resolv.conf` can hold a `search` list and `options ndots:N`. A name with fewer than N dots is tried with each search suffix first. In Kubernetes pods `ndots` is 5 by default, so a lookup for `api.example.com` (two dots) first tries `api.example.com.default.svc.cluster.local`, then `.svc.cluster.local`, and so on before the real name, causing several extra queries and delays when they time out. A trailing dot (`api.example.com.`) marks a fully qualified name and skips the search list; lowering `ndots` for the workload is another fix.

```
$ kubectl exec -it deploy/web -- cat /etc/resolv.conf
$ kubectl exec -it deploy/web -- getent hosts payments.prod.svc
$ kubectl -n kube-system logs deploy/coredns --tail=20
```

MARGA by EknathaLabs
marga.eknathalabs.com

## Split-horizon and private zones

Internal and external users may get different answers for the same name. A cloud private zone, a company resolver or a VPN can return private addresses that a public resolver would never return. When debugging, always test from the same network position as the failing client.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How DNS, TLS and PKI: debugging DNS failures fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Read the status** | NOERROR with an answer: the name resolved. | Treating SERVFAIL as "name does not exist". |
| **Test each hop** | The host's own view: getent hosts name uses the same libraries as applications, including /etc/hosts and search domains. | Testing only with a public resolver. |
| **Search domains and ndots** | /etc/resolv.conf can hold a search list and options ndots:N. | Ignoring the search list and ndots in containers. |
| **Split-horizon and private zones** | Internal and external users may get different answers for the same name. | Forgetting that private zones differ from public DNS. |

Flow: Read the status to Test each hop / Test each hop to Search domains and ndots / Search domains and ndots to Split-horizon and private zones

## Incident: Slow everything after a DNS change

> **Incident:** *After a platform upgrade, API calls from pods got slow, and the DNS servers showed ten times more queries than before.*

```
$ kubectl exec -it deploy/api -- sh -c 'time getent hosts payments.vendor.example.com'
real    0m5.012s
$ kubectl exec -it deploy/api -- cat /etc/resolv.conf
search prod.svc.cluster.local svc.cluster.local cluster.local corp.example.com
options ndots:5
```

An extra search domain from the company network had been added, and with `ndots:5` every external name was tried with four suffixes first. One suffix hit a slow forwarder that answered after five seconds. The team set `ndots:2` for the workloads that call outside, used fully qualified names in configs and deployed a node-local DNS cache. Query volume fell by 80 percent.

## Common mistakes

- Testing only with a public resolver
- Treating SERVFAIL as "name does not exist"
- Ignoring the search list and ndots in containers
- Forgetting that private zones differ from public DNS

### Check yourself

**What does NXDOMAIN mean?**

Answer: The name does not exist

**Which command shows what applications on the host see, including /etc/hosts?**

Answer: getent hosts name

**Why does ndots:5 cause extra lookups in Kubernetes?**

Answer: Names with fewer than five dots are tried with each search suffix first

**dig works from your laptop, but a pod fails to resolve an internal name. What is a likely cause?**

MARGA by EknathaLabs marga.eknathalabs.com

Answer: A private zone or split-horizon answer only visible from the cluster network

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 15. DNS, TLS and PKI: the TLS handshake and certificate chains

13 minutes. Follow a TLS connection from hello to encrypted data, read a certificate chain with openssl, and diagnose name mismatch, expired and incomplete-chain errors.

TLS errors have a reputation for being cryptic, yet a short list of causes explains most of them: the name does not match, the certificate is expired, the chain is incomplete or the client does not trust the issuer. openssl can show you which one in seconds.

## What you will learn

- The main steps of a TLS 1.3 handshake
- Leaf, intermediate and root certificates, and trust stores
- Subject Alternative Names and SNI
- Using `openssl s_client` to find the real error

## The handshake in short

The client sends a hello with the versions and ciphers it supports and the **SNI** (the server name it wants, so one IP can serve many sites). The server answers with its choice, its certificate chain and proof that it holds the private key. Both sides derive session keys, and encrypted data flows. TLS 1.3 completes this in one round trip and removes old, weak options. The client checks that the chain leads to a trusted root, that the dates are valid, that the name matches, and (if configured) that the certificate is not revoked.

## Chains and trust

A server certificate (the **leaf**) is signed by an **intermediate** CA, which is signed by a **root** CA that is in the client's trust store. The server must send the leaf and the intermediates; the root is already trusted by clients and is not sent. If the server leaves out an intermediate, some clients (browsers can fetch missing intermediates) succeed while others (curl, Java, Python, mobile apps) fail with "unable to get local issuer certificate".

```
$ openssl s_client -connect www.example.com:443 -servername www.example.com -showcerts </dev/null
$ openssl s_client -connect www.example.com:443 -servername www.example.com </dev/null 2>/dev/null \
     | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
$ curl -vI https://www.example.com 2>&1 | grep -i -E 'subject|issuer|expire|verify'
```

Always pass `-servername`, or you may get the default certificate instead of the right one. The output shows the chain (`s_client` lists each certificate with its depth), the validity dates and `Verify return code`. Code 0 means the chain is trusted.

## Names and SANs

Modern clients match the name against the **Subject Alternative Name** list, not the old Common Name. A certificate for `example.com` does not cover `www.example.com`, and a wildcard `*.example.com` covers one label only (`a.example.com`, not `a.b.example.com` and not the bare apex). Name mismatch errors usually mean the wrong certificate is being served for that hostname, often after a load balancer or SNI change.

## Time matters

Certificates have a not-before and a not-after date. A client with a wrong clock reports "not yet valid" or "expired" for a good certificate, so check `date` on the failing machine, especially on appliances and containers without time sync.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How DNS, TLS and PKI: the TLS handshake and... fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **The handshake in short** | The client sends a hello with the versions and ciphers it supports and the SNI (the server name it wants, so one IP can serve many sites). | Installing only the leaf certificate. |
| **Chains and trust** | A server certificate (the leaf) is signed by an intermediate CA, which is signed by a root CA that is in the client's trust store. | Forgetting SNI when testing with openssl. |
| **Names and SANs** | Modern clients match the name against the Subject Alternative Name list, not the old Common Name. | Assuming a wildcard covers the apex and deeper subdomains. |
| **Time matters** | Certificates have a not-before and a not-after date. | Testing with a browser only. |

Flow: The handshake in short to Chains and trust / Chains and trust to Names and SANs / Names and SANs to Time matters

## Incident: Works in Chrome, fails in the payment gateway

**Incident:** *After a certificate renewal, customers could use the website but a partner's server-to-server calls started failing.*

```
$ openssl s_client -connect pay.example.com:443 -servername pay.example.com </dev/null 2>&1 | grep \
     -E 'depth|Verify'
depth=0 CN = pay.example.com
verify error:num=20:unable to get local issuer certificate
Verify return code: 21 (unable to verify the first certificate)
```

The renewal script had installed only the leaf file, not the full chain. Browsers hid the problem by fetching the intermediate, while the partner's Java client could not. The team changed the deployment to use the full-chain file, added a post-deploy check that runs `openssl s_client` with strict verification from outside, and monitored the chain depth.

## Common mistakes

- Installing only the leaf certificate
- Testing with a browser only
- Forgetting SNI when testing with openssl
- Assuming a wildcard covers the apex and deeper subdomains

### Check yourself

**What does SNI tell the server?**

Answer: The hostname the client wants, so it can present the right certificate

**Which certificates must a server send?**

Answer: The leaf and the intermediate certificates

**Does *.example.com match a.b.example.com?**

Answer: No, a wildcard covers one label only

**A good certificate is reported as expired on one server only. What do you check?**

Answer: The clock on that machine

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 16. DNS, TLS and PKI: certificate lifecycle and automation

12 minutes. Follow a certificate from key and CSR to issuance, validation, renewal and revocation, and automate it with ACME so expiry never depends on a person remembering.

Certificates expire by design. Expiry limits the damage from a leaked key, but it creates a recurring job. Teams that renew by hand eventually forget one. The cure is to treat certificates as short-lived, automated objects and to watch the dates.

## What you will learn

- The steps from private key to trusted certificate
- How ACME challenges prove control of a name
- Renewal, rotation and revocation basics
- How to monitor expiry so nothing is a surprise

## From key to certificate

1. Generate a **private key** and keep it on the server (or in an HSM or key service). It never leaves.
2. Create a **CSR** (certificate signing request) containing the public key and the names.
3. The **CA** verifies you control the names, then signs a certificate.
4. Install the certificate with its chain, and reload the service.

```
$ openssl req -new -newkey rsa:2048 -nodes -keyout app.key -out app.csr -subj "/CN=app.example.com \
    " -addext "subjectAltName=DNS:app.example.com"
$ openssl req -in app.csr -noout -text | grep -A1 "Subject Alternative"
```

Protect the key with file permissions (`chmod 600`) and ownership. A new key at each renewal limits the time a stolen key is useful.

## ACME and automatic validation

ACME (RFC 8555) is the protocol used by Let's Encrypt and other CAs. The client proves control of a name with a challenge:

- **HTTP-01**: place a token at `/.well-known/acme-challenge/` on port 80 for that name. Simple, but needs public reachability and cannot issue wildcards.
- **DNS-01**: create a TXT record `_acme-challenge.name`. Works for wildcards and for internal servers, and needs API access to your DNS, which should be scoped narrowly.

Clients such as certbot, acme.sh, Caddy and cert-manager renew automatically, typically when about a third of the lifetime remains. Public certificates are valid for a few months at most, and the industry is moving to even shorter lifetimes, so automation is not optional.

## Rotate and revoke

Renewal replaces the certificate; the service must **reload** it, or it keeps serving the old one until restart. Test that reload in staging. **Revocation** tells clients a certificate must not be trusted before it expires (CRL lists and OCSP queries), but client support is uneven, so short lifetimes are the stronger control. If a private key leaks, replace the key, issue a new certificate, revoke the old one and look for how it leaked.

## Watch the dates

Automation fails silently: a DNS credential expires, a firewall blocks port 80, a reload hook breaks. Probe the real endpoints from outside and alert well before expiry.

```
groups:
  - name: certificates
```

MARGA by EknathaLabs marga.eknathalabs.com

```
rules:
  - alert: CertificateExpiresSoon
    expr: probe_ssl_earliest_cert_expiry - time() < 14 * 86400
    for: 10m
    labels:
      severity: page
    annotations:
      summary: "Certificate for {{ $labels.instance }} expires in under 14 days"
```

This alert uses the Prometheus blackbox exporter's probe metric. With renewal at about 30 days remaining, a 14-day warning means the automation has already failed twice.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How DNS, TLS and PKI: certificate lifecycle and... fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **From key to certificate** | Generate a private key and keep it on the server (or in an HSM or key service). | Manual renewal with a calendar reminder. |
| **ACME and automatic validation** | ACME (RFC 8555) is the protocol used by Let's Encrypt and other CAs. | Monitoring only public sites and not internal ones. |
| **Rotate and revoke** | Renewal replaces the certificate; the service must reload it, or it keeps serving the old one until restart. | Renewing the file but not reloading the service. |
| **Watch the dates** | Automation fails silently: a DNS credential expires, a firewall blocks port 80, a reload hook breaks. | Watch the dates Automation fails silently: a DNS credential expires, a firewall blocks port 80, a reload hook breaks. |

Flow: From key to certificate to ACME and automatic validation / ACME and automatic validation to Rotate and revoke / Rotate and revoke to Watch the dates

## Incident: The forgotten certificate

> **Incident:** *An internal tool that every engineer used for deployments stopped working on a Monday morning because its certificate had expired over the weekend.*

```
$ echo | openssl s_client -connect deploy.corp.example:443 -servername deploy.corp.example 2>/dev/ \
    null | openssl x509 -noout -enddate
notAfter=Mar  2 06:00:00 2026 GMT
```

The certificate had been created by hand two years earlier by someone who had since left, and it was not in any inventory or monitor. The team issued a new one, then created an inventory from DNS and load balancers, moved every endpoint to automated issuance and added the expiry alert above for all of them.

## Common mistakes

- Manual renewal with a calendar reminder
- Renewing the file but not reloading the service
- Monitoring only public sites and not internal ones
- Reusing the same private key forever

### Check yourself

**What does a CSR contain?**

Answer: The public key and the requested names, signed with the private key

**Which ACME challenge works for wildcard certificates?**

Answer: DNS-01

MARGA by EknathaLabs marga.eknathalabs.com

**Why alert on expiry 14 days ahead when renewal starts earlier?**

Answer: If the alert fires, the automation has already failed and needs a person

**What must happen after a certificate file is renewed?**

Answer: The service must reload it

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 17. DNS, TLS and PKI: mTLS and internal certificate authorities

13 minutes. Use mutual TLS for service identity, run an internal CA the safe way with cert-manager, and distribute trust bundles and rotation without outages.

Inside a company, services still need to know who they are talking to. Mutual TLS (mTLS) gives both sides a certificate, so the server verifies the client as well as the client verifying the server. The hard parts are not the cryptography but trust distribution, short lifetimes and rotation.

## What you will learn

- What mTLS adds to normal TLS and where it is used
- Designing an internal CA hierarchy with an offline root
- Issuing and renewing certificates in Kubernetes with cert-manager
- Rotating trust bundles and CA certificates safely

## Why mTLS

Normal TLS proves the server to the client. With mTLS the server requests a client certificate and checks it against a trusted CA. The certificate subject or SAN (often a service name or a SPIFFE ID such as `spiffe://prod/ns/shop/sa/checkout`) becomes the **identity** used in authorization rules. Service meshes use it for encryption and identity between pods, and some APIs and databases require it for clients.

## A sensible CA design

Keep the **root CA key offline** (or in an HSM) and use it only to sign a few **intermediate** CAs. Issuing intermediates sign day-to-day certificates and can be replaced or revoked without replacing the root. Use short lifetimes (days or hours for workloads) so that revocation is rarely needed. Separate intermediates by environment so a problem in staging cannot sign production identities.

```
$ openssl x509 -in client.crt -noout -subject -issuer -dates -ext subjectAltName
$ openssl verify -CAfile ca-bundle.pem client.crt
$ curl --cert client.crt --key client.key --cacert ca-bundle.pem https://internal-api.prod.svc:844 \
    3/health
```

## cert-manager in Kubernetes

cert-manager watches `Certificate` objects, asks an **Issuer** (a CA, Vault, ACME or a cloud service) to sign them, stores the result in a Secret and renews before expiry.

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: checkout-mtls
  namespace: shop
spec:
  secretName: checkout-tls
  duration: 24h
  renewBefore: 8h
  commonName: checkout.shop.svc
  dnsNames:
    - checkout.shop.svc
    - checkout.shop.svc.cluster.local
  issuerRef:
    name: internal-ca
    kind: ClusterIssuer
  usages:
    - server auth
    - client auth
```

Applications must reload the changed Secret (watch the files or restart through a controller). A 24-hour certificate that is never reloaded becomes an outage every day.

MARGA by EknathaLabs marga.eknathalabs.com

## Trust bundles and rotation

Clients need the CA certificates they should trust, the **trust bundle**. To rotate a CA safely, first distribute the *new* CA certificate to every client (bundle contains old and new), then start issuing from the new CA, then wait until all old certificates have expired, and finally remove the old CA. Doing it in this order means there is no moment when a valid certificate is untrusted. CA certificates have expiry dates too, so put them on the same calendar and alerts as leaf certificates.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How DNS, TLS and PKI: mTLS and internal certificate... fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Why mTLS** | Normal TLS proves the server to the client. | Keeping the root CA key online on a shared server. |
| **A sensible CA design** | Keep the root CA key offline (or in an HSM) and use it only to sign a few intermediate CAs. | Replacing a CA without a trust bundle overlap. |
| **cert-manager in Kubernetes** | cert-manager watches Certificate objects, asks an Issuer (a CA, Vault, ACME or a cloud service) to sign them, stores the result in a Secret and renews before expiry. | Forgetting that the CA certificate itself expires. |
| **Trust bundles and rotation** | Clients need the CA certificates they should trust, the trust bundle. | Issuing long-lived workload certificates. |

Flow: Why mTLS to A sensible CA design / A sensible CA design to cert-manager in Kubernetes / cert-manager in Kubernetes to Trust bundles and rotation

## Incident: The CA that expired on a Sunday

> **Incident:** *At 03:00 on a Sunday all service-to-service calls failed with "certificate has expired", although every leaf certificate had been renewed that day.*

```
$ openssl x509 -in /etc/pki/internal-ca.crt -noout -enddate
notAfter=Jun 14 03:00:00 2026 GMT
```

The intermediate CA certificate, created years earlier with a long but finite lifetime, had expired. The leaf certificates were fine, but their issuer was not. Nobody monitored CA expiry, since renewal automation covered only leaves. The team issued a new intermediate, pushed a trust bundle with both CAs, restarted services in waves, and added CA certificates to the expiry inventory with alerts at 90, 30 and 7 days.

## Common mistakes

- Keeping the root CA key online on a shared server
- Issuing long-lived workload certificates
- Replacing a CA without a trust bundle overlap
- Forgetting that the CA certificate itself expires

### Check yourself

**What does mTLS add compared with normal TLS?**

Answer: The client also presents a certificate that the server verifies

**Why keep the root CA key offline?**

Answer: A stolen root key would let an attacker mint trusted certificates for everything

**What is the safe order when rotating an internal CA?**

Answer: Distribute the new CA, issue from it, let old certificates expire, then remove the old CA

MARGA by EknathaLabs marga.eknathalabs.com

**What must an application do when cert-manager renews its Secret?**

Answer: Reload the new certificate files

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 18. DNS, TLS and PKI: outage prevention checklist

11 minutes. Prevent the common DNS and certificate outages with an inventory, expiry and domain-renewal alerts, safe change steps, redundancy and a rehearsed rollback.

A large share of avoidable outages comes from the same few causes: an expired certificate, an expired domain, a careless DNS change, or one DNS provider that went down. Each has a simple preventive practice. This lesson collects them into a checklist you can adopt.

## What you will learn

- Build and maintain an inventory of names and certificates
- Alert on certificate, CA and domain expiry
- Change DNS safely, with review, TTL planning and rollback
- Add redundancy for DNS itself

## Know what you have

You cannot protect what you cannot list. Keep an inventory of domains (and who owns the registrar account), DNS zones and providers, public and internal certificates, CA certificates and the systems that serve them. Generate it from sources of truth (DNS zones, load balancers, ingress objects, certificate stores) instead of a spreadsheet that goes stale, and check for names pointing at services that no longer exist, which are also a takeover risk.

## Alert before it hurts

- **Certificates**: probe from outside and alert at 30, 14 and 7 days; a page at 7.
- **CA and signing certificates**: 90, 30 and 7 days.
- **Domains**: registrar expiry at 60 and 30 days, with auto-renew on, a valid payment method and a shared role mailbox (not a leaver's address) as contact.
- **DNS health**: synthetic lookups of key names from several networks, and an alert when the answer changes unexpectedly.

```
$ whois example.com | grep -i -E 'expir|registrar'
$ dig +short NS example.com
$ echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x \
    509 -noout -enddate
```

## Change DNS like code

Store zone data as code (Terraform or a similar tool), review changes, and apply from a pipeline. Before a risky change: lower the TTL a day ahead, write the exact rollback record, check that the new target works by testing it directly (for example with `curl --resolve`), change one thing at a time and compare answers from the authoritative server and public resolvers afterward. Protect critical zones with registry locks and restricted write access, since a hijacked DNS record is a hijacked service.

```
$ curl --resolve app.example.com:443:198.51.100.20 https://app.example.com/health
```

## Make DNS itself redundant

Use at least two authoritative servers on **separate networks**, and for critical domains consider two independent providers with synchronized zones. Keep delegation (NS and glue records) consistent. Know how your own systems behave when DNS is down: clients should cache sensibly, applications should not do blocking lookups on every request, and the tools you need during an incident (status page, access to consoles) must not depend only on the domain that is broken.

MARGA by EknathaLabs
marga.eknathalabs.com

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How DNS, TLS and PKI: outage prevention checklist fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Know what you have** | You cannot protect what you cannot list. | No inventory, so nobody knows what expires when. |
| **Alert before it hurts** | Certificates: probe from outside and alert at 30, 14 and 7 days; a page at 7. | Registrar contact tied to one person's mailbox. |
| **Change DNS like code** | Store zone data as code (Terraform or a similar tool), review changes, and apply from a pipeline. | DNS changes made in a console with no review or rollback. |
| **Make DNS itself redundant** | Use at least two authoritative servers on separate networks, and for critical domains consider two independent providers with synchronized zones. | A single provider and a single point of failure for DNS. |

Flow: Know what you have to Alert before it hurts / Alert before it hurts to Change DNS like code / Change DNS like code to Make DNS itself redundant

## Incident: The domain that expired

> **Incident:** *A company's main domain stopped resolving on a Friday evening, taking down the website, email and the single sign-on service.*

```
$ whois company.example | grep -i expir
Registry Expiry Date: 2026-06-05T00:00:00Z
$ dig +short NS company.example
```

The renewal reminders had gone to the mailbox of an employee who had left, and the payment card on file had expired. The registrar kept the domain in a grace period, and a manager had to find the account and pay by phone before DNS returned. The company moved the domain to a role-based account with several admins, enabled auto-renew, added expiry alerts and registry lock, and wrote a one-page recovery runbook for domain loss.

## Common mistakes

- No inventory, so nobody knows what expires when
- Registrar contact tied to one person's mailbox
- DNS changes made in a console with no review or rollback
- A single provider and a single point of failure for DNS

### Check yourself

**Why should registrar contacts be role mailboxes?**

Answer: Reminders and recovery must not depend on one person

**What is a safe way to test a new DNS target before changing the record?**

Answer: Use curl --resolve to reach the new address with the real hostname

**What does a second DNS provider protect against?**

Answer: An outage or failure at one provider

**When should you lower a record's TTL before a planned move?**

Answer: At least one old-TTL period ahead of the change

MARGA by EknathaLabs marga.eknathalabs.com

# Cheat sheet: Networking

Addressing, DNS, HTTP and TLS checks, and the order to troubleshoot connectivity.

## Inspect your host

```
ip -br addr                          # interfaces and addresses
ip route                             # routing table
ip route get 10.20.20.15             # which route a destination uses
ss -tulpn                            # listening TCP/UDP ports and owners
ss -s                                # socket summary
```

## CIDR quick reference

| CIDR | Addresses | Mask |
|---|---|---|
| /32 | 1 | 255.255.255.255 |
| /30 | 4 | 255.255.255.252 |
| /28 | 16 | 255.255.255.240 |
| /24 | 256 | 255.255.255.0 |
| /20 | 4,096 | 255.255.240.0 |
| /16 | 65,536 | 255.255.0.0 |
| /8 | 16,777,216 | 255.0.0.0 |

Private ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. Cloud providers reserve a few addresses in every subnet.

## DNS

```
dig +short example.com A
dig +short example.com MX
dig @8.8.8.8 example.com             # ask a specific resolver
dig +trace example.com               # follow delegation from the root
dig -x 192.0.2.10                    # reverse lookup
getent hosts example.com             # what the system resolver returns
```

| Record | Holds |
|---|---|
| A / AAAA | IPv4 / IPv6 address |
| CNAME | Alias to another name |
| MX | Mail servers |
| TXT | Free text (SPF, verification) |
| NS | Name servers for the zone |

## HTTP and TLS

```
curl -sS -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com
curl -I https://example.com
curl -v --resolve example.com:443:192.0.2.10 https://example.com
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null | openssl \
     x509 -noout -dates -subject
```

| Status | Meaning |
|---|---|
| 2xx | Success |
| 301 / 302 | Redirect (permanent / temporary) |
| 401 / 403 | Not authenticated / not allowed |
| 404 | Not found |
| 429 | Rate limited |
| 502 / 503 / 504 | Bad gateway / unavailable / gateway timeout |

MARGA by EknathaLabs
marga.eknathalabs.com

## Connectivity tests

```
ping -c 3 192.0.2.10                # reachability
traceroute -n 192.0.2.10            # path
mtr -rwc 20 192.0.2.10              # path with loss per hop
nc -zv -w 3 192.0.2.10 5432         # is a TCP port open
tcpdump -ni eth0 port 53 -c 20      # capture a few DNS packets
```

## Troubleshooting order

1. Name resolves to the expected address? (`dig`, `getent hosts`)
2. Route exists? (`ip route get`)
3. Host reachable? (`ping`, `mtr`)
4. Port open? (`nc -zv`) - timeout means dropped, refused means nothing listening
5. Firewall rules allow it, in both directions where stateless?
6. TLS valid and trusted? (`openssl s_client`)
7. Application responds correctly? (`curl -v`)

## Firewall quick reference

```
iptables -S                         # list rules
nft list ruleset                    # nftables rules
ufw status verbose
firewall-cmd --list-all
```

## Name resolution and routing

```
dig example.com A +short
dig @1.1.1.1 example.com MX
dig +trace example.com
dig -x 93.184.216.34 +short
resolvectl status
getent hosts example.com
nslookup example.com
ip route get 8.8.8.8
ip neigh
mtr -rwzc 20 example.com
```

> **Warning:** *DNS answers are cached. After you change a record, old answers live until the TTL expires, so lower the TTL before a planned change, not during it.*

## Connectivity and ports

```
nc -zv db.internal 5432
nc -l 8080
curl -v --connect-timeout 5 https://example.com
curl -I https://example.com
curl --resolve example.com:443:10.0.0.10 https://example.com/healthz
openssl s_client -connect example.com:443 -servername example.com </dev/null | openssl x509 -noout \
     -dates -subject
ss -ltnp
ss -tn state established '( dport = :443 )' | wc -l
nmap -p 22,80,443 10.0.0.10
```

> **Warning:** *Scan only hosts and networks that you own or have written permission to test.* `nmap` *against other networks can break rules and trigger security alarms.*

> **Warning:** `curl -k` *and* `--insecure` *turn off certificate checks. Use them only to find a problem, never in scripts.*

MARGA by EknathaLabs
marga.eknathalabs.com

## Firewall and packet capture

```
sudo iptables -S
sudo nft list ruleset
sudo ufw status verbose
sudo firewall-cmd --list-all
sudo tcpdump -ni any host 10.0.0.10 and port 443 -c 50 -w /tmp/cap.pcap
sudo tcpdump -nn -r /tmp/cap.pcap | head
ip -s link show eth0
ethtool eth0
```

**Warning:** *Changing firewall rules over SSH can lock you out. Add an allow rule for your own session first, keep a second session open, and schedule an automatic rollback.*

**Danger:** *Packet captures can contain passwords, tokens and personal data. Store them securely and delete them when the work is done.*

## Cautions for network changes

**Warning:** *Changing an IP address, route or MTU on a remote host with* `ip addr`, `ip route` *or* `ip link set` *can cut your connection. Use the console or an out-of-band path for risky changes.*

**Warning:** `ip route add default` *with a wrong gateway breaks all outbound traffic. Check* `ip route` *before and after.*

**Warning:** *Overlapping CIDR ranges between VPCs, VPNs and clusters cannot be peered later. Plan address space before you create networks.*

**Warning:** *Disabling reverse path filtering or turning on* `ip_forward` *changes how the host handles traffic. Do it only on purpose and document it.*

MARGA by EknathaLabs
marga.eknathalabs.com